MCP-Lock

Provenance and tool-definition pinning to stop AI coding agents from silently running unreviewed tool servers



Domain: Cybersecurity · Software supply-chain security · AI agents (Model Context Protocol)

Team member	Register number
Aakash G S	RA2311042010060
Hariharan J	RA2311042010059
J Jaswanth Raj	RA2311042010013

Project guide

Sai Prasanth D

Assistant Professor, Department of Data Science and Business Systems

Department of Data Science and Business Systems, School of Computing, SRM Institute of Science and Technology, Kattankulathur

21CSP401L Major Project · Review 0

Problem statement and background

Criterion 1

Background: AI coding agents (Claude Code, Cursor, …) get their tools from MCP servers, which they start straight from npm or PyPI.

1 Config says

`npx -y weather-mcp`

No version pinned: “give me the latest”

2 Agent

starts the server

Automatically, every session

3 Registry

serves newest version

May be hijacked or changed since review

4 Server runs

+ tools reach the model

Tool descriptions act as instructions

PROBLEM STATEMENT

Unpinned MCP-server launches let the code and the tool descriptions an agent runs change after a human reviewed them, with no signal to the user. Existing pre-install checks look only at package names, sources and age. **We need a gate that, before every launch and without executing package code, detects when the version about to run departs from the reviewed one in build provenance, publisher, install scripts or tool definitions,** and that catches more real attacks than existing checks at a measured false-block cost.

MCP = Model Context Protocol, the open standard through which agents call external tools. Rug pull: a server changes a tool after it was approved.

Motivation and justification: why now

Criterion 1

9.8%

of 2,660 real agent setups launch MCP servers unpinned [1]

500+

npm packages hit by the Shai-Hulud worm through stolen maintainer tokens, Sept. 2025 [2]

36.5%

of 2,000 sampled MCP packages changed their tools within 90 days (our pilot census)

Rarely

do coding assistants check trust signals before installing anything [3]

Who benefits

- **Developers using AI coding agents:** their keys, files and tokens stay safe
- **Companies and security teams:** one auditable lockfile for every approved tool server
- **Open-source maintainers and registries:** a hijacked release no longer spreads silently

Scope

In: MCP servers launched from npm and PyPI; attackers who squat a name, steal a token, enter the release pipeline, own the package, or own a dependency.

Out: compromise of the registry itself, forged Sigstore signatures, attacks on the language model.

The 36.5% figure is from our RQ1 census (stratified random sample, tool definitions extracted statically). Numbers in [ ] refer to the References slide.

Literature survey

Criterion 2

Ref	Author, year, venue	Method / contribution	Limitation (what is left open)
[4]	**Ladisa et al., 2023 IEEE S&P**	SoK: taxonomy of attacks on open-source supply chains, incl. account takeover and install scripts	General package ecosystems; nothing on agents, MCP or tool descriptions
[5]	**Hou et al., 2025 arXiv**	Survey of the MCP landscape; defines tool poisoning and rug-pull threats	Survey only: no defence measured on real packages
[6]	**Bhatt et al., 2025 arXiv**	ETDI: OAuth-signed, versioned tool definitions with policy-based access control	Needs servers and registries to change and sign; not usable with existing unsigned packages
[7]	**Bagmar & Saraf, 2026 arXiv**	Pre-install hook for coding agents: 7 checks on name, source and age (pip)	Never compares with the version you reviewed; misses hijacks of existing packages
[1]	**Kapner et al., 2026 arXiv**	Scans 2,660 coding-agent setups; finds 9.8% with unpinned MCP servers	Measures exposure only; no provenance analysis and no defence
[8]	**Kraishan, 2026 arXiv**	Census of drift in MCP-registry fields over 8,900 multi-version servers	Registry does not version tool definitions, so tool changes go unmeasured; recommends client-side pinning

Common thread: no work checks, before launch, whether a server's build, publisher, install scripts and tools still match what was reviewed.

All six sources verified against their primary pages (docs/CITATIONS_VERIFIED.md). Ladisa et al. appeared at IEEE S&P 2023, DOI 10.1109/SP46215.2023.10179304.

Research gap, market and patent study

Criterion 2

Existing solution	Before launch, runs no code	Pins tool definitions	Build, publisher, script continuity	No server / registry change
Bagmar & Saraf hook [7]	✓	✗	✗	✓
MCP-Scan, mcp-pin, mcpseal [9]–[11]	✗	✓	partly	✓
Signed definitions: ETDI [6]	–	✓	–	✗
npm provenance, Sigstore [12], [13]	✓	✗	partly	✓
MCP-Lock (proposed)	✓	✓	✓	✓

Patent landscape

US 12,314,394 B2 (Socket, 2025) [14]:
scans a project's dependency files for malware via a cache of scanned packages. No comparison with a reviewed version.

US 10,055,576 B2 (Red Hat, 2018) [15]:
blocks repository packages whose components are on an insecure list. Known-bad lists only.

No patent found that pins agent tool definitions before launch.

G1 No measurement of how often MCP tool definitions really change

G2 No check reads tool definitions before launch, without starting the server

G3 No continuity checks on build, publisher and install scripts per server

G4 No evaluation on real attacks with the false-block cost on honest upgrades

Our edge

One lockfile pins archive, provenance, publisher, install scripts and tool definitions; the gate checks all five before every launch, needs no ecosystem change, and closes G1–G4.

Market: Socket and npm provenance protect packages, MCP-Scan and the pinning proxies protect tools after start-up. Patent search: Google Patents, 3 Nov. 2026, not exhaustive.

Objectives (SMART)

Criterion 3

O1 Measure the problem

Census MCP packages on npm and PyPI; extract tool definitions from a stratified sample of 2,000 and report the 90-day change rate with 95% intervals.

Closes G1

Done: 15,272 packages; 36.5% changed tools

O2 Build MCP-Lock

A lockfile and a launch gate with 5 checks (S1–S5) that never runs package code, installable as a coding-agent hook and covered by automated tests.

Closes G2, G3

Done: gate + hook, 103 tests

O3 Prove it works

Pre-registered test on ≥ 490 real attacks: detect more than the baseline (McNemar, $\alpha = 0.05$); report false blocks on 8,180 honest upgrades; withdraw the claim if S2–S5 add < 2.

Closes G4

Done: 39.8% vs 19.6%, +0.42 pp

O4 Stress-test and publish

Build 12 adaptive attacks, add hardening with measured costs, release code and data, and submit an IEEE paper before the final review.

Closes G2–G4

Planned: paper drafted, to submit

S specific task · M a number or test · A sized to a 3-person team · R tied to gap G1–G4 · T each closes at a review milestone (see work plan).

Alignment with UN Sustainable Development Goals

Criterion 4

9

SDG 9

Industry, Innovation and Infrastructure

9.1 Reliable, resilient infrastructure: the open-source supply chain is digital infrastructure; MCP-Lock keeps one hijacked release from reaching every agent that launches it.

9.5 Upgrade technological capability: a new, pre-registered security method with public code.

16

SDG 16

Peace, Justice and Strong Institutions

16.a Capacity to prevent crime: blocks the cyber-crime pattern behind worms like Shai-Hulud (stolen tokens, install-script payloads).

16.6 Accountable, transparent systems: every verdict is traceable to an auditable lockfile.

17

SDG 17

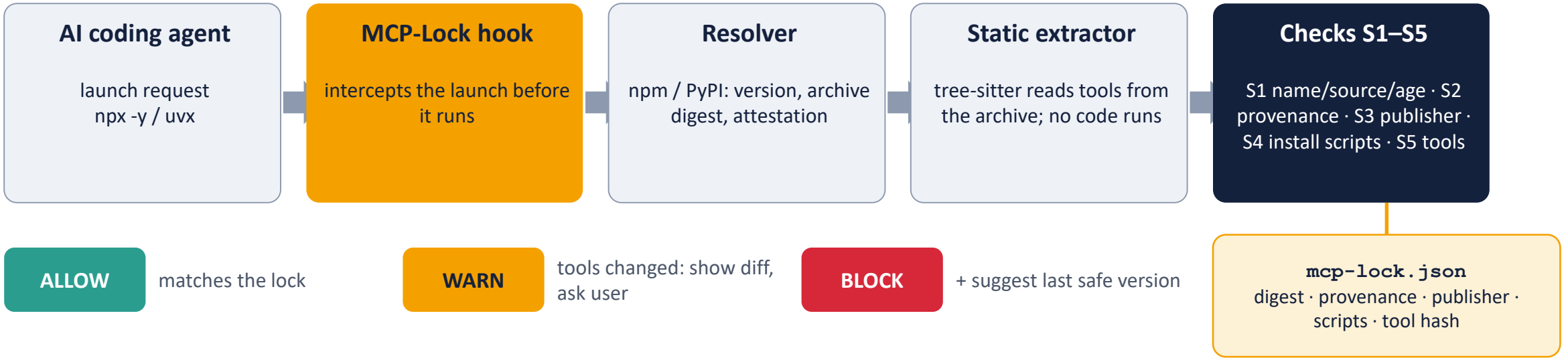
Partnerships for the Goals

17.6 Knowledge sharing in science and technology: code, data, replay bundles and the OSF pre-registration are open, so others, including developing-country teams, can reuse and verify them.

Targets as defined in UN resolution A/RES/70/1, "Transforming our world: the 2030 Agenda for Sustainable Development" [17].

Proposed methodology and high-level architecture

Criteria 3 & 5



Modules

`census/`	RQ1: frame of MCP packages, version history, static tool extraction
`lockfile/`	lock format spec and canonical tool-definition hashing
`gate/`	checks S1–S5, verdicts, safe-version suggestions, agent hook, CLI
`eval/`	replay of 490 attacks and 8,180 upgrades; adaptive attacks in Docker

Tech stack

- Python 3.11
- tree-sitter (JS/TS)
- npm & PyPI APIs
- Sigstore / SLSA provenance
- Docker sandbox
- pytest + GitHub Actions
- Claude Code hook
- OSF · LaTeX IEEEtran

Method: lock once after review (mcplock lock), then the gate checks every launch. Evaluation is pre-registered on OSF [16] against the baseline hook [7].

Work plan and review timeline

Criterion 5



Done dates from the project's git history. Review dates follow the department calendar.

References

Criteria 2 & 5

[1] B. Kapner, C. Soceanu, A. Petrunin, and H. Gartner, “Scanning the harness: Configuration exposures in AI coding-agent supply chains,” arXiv:2609.07360v3, 2026.

[2] CISA, “Widespread supply chain compromise impacting npm ecosystem,” Alert, Sept. 23, 2025.

[3] P. Shan, “Do AI coding assistants check before they install? A pre-registered demand-side audit of trust signals in the research software supply chain,” arXiv:2609.07754, 2026.

[4] P. Ladisa, H. Plate, M. Martinez, and O. Barais, “SoK: Taxonomy of attacks on open-source software supply chains,” in Proc. IEEE Symp. Security and Privacy (SP), 2023, pp. 1509–1526, doi: 10.1109/SP46215.2023.10179304.

[5] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model Context Protocol (MCP): Landscape, security threats, and future research directions,” arXiv:2503.23278, 2025.

[6] M. Bhatt, V. S. Narajala, and I. Habler, “ETDI: Mitigating tool squatting and rug pull attacks in Model Context Protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control,” arXiv:2506.01333, 2025.

[7] A. Bagmar and P. Saraf, “Setup complete, now you are compromised: Weaponizing setup instructions against AI coding agents,” arXiv:2607.15143, 2026.

[8] O. Kraishan, “Same name, different server: A security census of silent drift in the Model Context Protocol ecosystem,” arXiv:2609.14119, 2026.

[9] L. Beurer-Kellner and M. Fischer, “Introducing MCP-Scan: Protecting MCP with Invariant,” Invariant Labs blog, Apr. 2025.

[10] GautamTalksDev, “mcp-pin: Pin what your agent approved; block it when it changes,” GitHub, 2026. [Online]. Available: github.com/GautamTalksDev/mcp-pin

[11] confuseddude, “mcpseal: Pins the hash of every approved MCP tool definition,” GitHub, 2026. [Online]. Available: github.com/confuseddude/mcpseal

[12] npm, “Generating provenance statements,” npm Docs, 2026. [Online]. Available: docs.npmjs.com/generating-provenance-statements

[13] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in Proc. ACM CCS, 2022, pp. 2353–2367, doi: 10.1145/3548606.3560596.

[14] F. H. Aboukhadijeh and M. C. Lysenko, “Detecting suspected malware in open-source dependencies,” U.S. Patent 12,314,394 B2, May 27, 2025.

[15] S. B. Milner and J. R. Bowes, “Detection of malicious software packages,” U.S. Patent 10,055,576 B2, Aug. 21, 2018.

[16] A. G S, “MCP-Lock: Does provenance and tool-definition pinning catch MCP-server supply-chain attacks that name/source checks miss, and at what false-block cost?” OSF Registries, Oct. 2026, doi: 10.17605/OSF.IO/2AMJ3.

[17] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, Oct. 2015.

Code and data: github.com/ftaakash/MCP-Lock · All entries checked against their primary source (docs/CITATIONS_VERIFIED.md); patents checked on Google Patents.